

Master Of Science

Department of Electrical and Computer Engineering

ECE 506 FORMAL PROJECT

Formal Property Verification (FPV) and Automatic Extracted Property (AEP) Analysis of a 4-Port Round-Robin Arbiter

INSTRUCTOR

Dr. Xiaoyu Song

FALL 2026

PROJECT

Proposal

On my honor, I have neither given nor received unauthorised aid on this project

Krishnavardhan Raju Godugu PSU ID: 904179101

1. Project Overview and Motivation

Arbitration logic serves as the backbone of modern System-on-Chip (SoC) interconnects, bus protocols (such as AXI4-Lite), and Network-on-Chip (NoC) routers, managing how multiple masters access a shared hardware resource. When verifying an arbiter using traditional simulation-based testbenches, stimulus generation often fails to hit complex multi-cycle corner cases. Subtle hardware bugs—such as combinational feedback loops, priority-masking glitches, deadlocks, and livelocks (request starvation)—frequently escape directed and constrained-random tests.

In this project, I will design and formally verify a **4-Port Round-Robin Arbiter** using **SystemVerilog Assertions (SVA)**. Instead of relying on individual simulation vectors, I will model the arbiter as a finite Kripke structure and apply mathematical model checking and reachability analysis to exhaustively explore all valid state transitions. Using **Synopsys VC Formal** and **Cadence JasperGold**, I will perform **Automatic Extracted Property (AEP)** analysis to catch structural RTL hazards and **Formal Property Verification (FPV)** to mathematically prove both **Safety** (mutual exclusion, valid grant handshaking, deadlock freedom) and **Liveness** (starvation freedom via Linear Temporal Logic) properties.

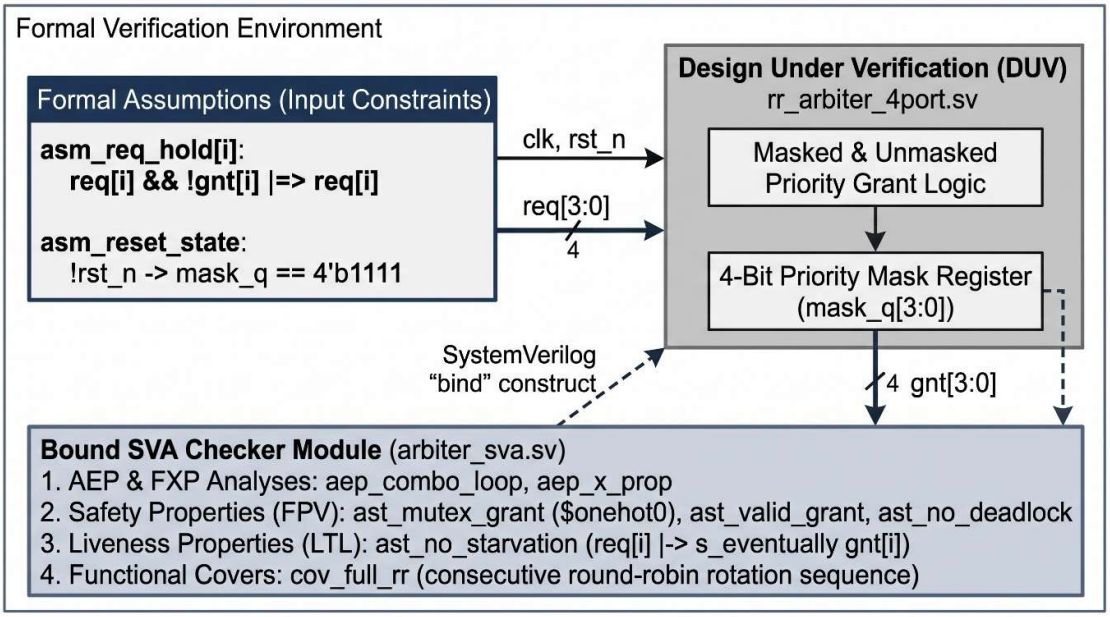
2. Design Under Verification (DUV) and Block Diagram

The Design Under Verification (DUV) is a synthesizable 4-port Round-Robin Arbiter (rr_arbiter_4port.sv) written in SystemVerilog. The module arbitrates among four competing masters asserting a 4-bit request vector (req[3:0]) and outputs a one-hot 4-bit grant vector (gnt[3:0]).

2.1 Interface Specification

- clk: System clock (rising-edge triggered).
- rst_n: Active-low synchronous reset.
- req[3:0]: 4-bit input request vector from Master 0 through Master 3.
- gnt[3:0]: 4-bit output grant vector indicating which master currently has access to the shared resource.

2.2 Architecture and Formal Verification Environment Block Diagram



2.3 Formal Kripke Structure and Reachability Model

To enforce rotating priority, the arbiter maintains a 4-bit internal mask register (mask_q[3:0]) that masks out recently serviced ports. Formally, the system is represented as a Kripke structure $M = (S, S_0, R, L)$, where S represents all possible valuations of the state register mask_q[3:0] and inputs req[3:0], S_0 is the initial reset state (mask_q == 4'b1111, gnt == 4'b0000), and R is the cycle-by-cycle state transition relation. The model checker performs state reachability analysis across M to verify that no reachable path violates safety invariants ($G\neg$fail) or traps an active request in an unfair starvation loop ($G(\text{req}_i \rightarrow F\text{gnt}_i)$).

3. Measurable Goal, Baseline, Planned Improvement, and Trade-off

To evaluate the effectiveness of my formal verification methodology and RTL improvements, this project establishes four concrete pillars:

3.1 Measurable Goal

My primary measurable target is to increase the formal property proof convergence rate from 50% (6 out of 12 properties passing) in the baseline design to 100% (12 out of 12 unbounded proofs passing) across all AEP, Safety, and Liveness checks, while achieving 100% assertion and functional cover reachability with zero deadlock or livelock counterexamples.

3.2 The Baseline

Before applying architectural fixes, I will establish a baseline using an initial **Baseline 4-Port Arbiter (**arbiter_baseline.sv**)** evaluated against the complete SVA property suite. This baseline design represents a naive implementation containing three critical hardware bugs:

1. **Combinational Feedback Loop:** The combinatorial grant logic directly feeds back into an unregistered mask wire within the same always block, creating a combinational loop that causes tool warnings and oscillation hazards.
2. **Deadlock on Mask Exhaustion:** When Port 3 (the highest-indexed port) is granted, the priority mask updates to 4'b0000. In the subsequent cycle, the baseline design fails to bypass the empty mask, causing the arbiter to output gnt == 4'b0000 even when requests (req != 4'b0000) are active resulting in a **deadlock**.
3. **Livelock / Request Starvation:** When Master 0 (req[0]) continuously asserts its request back-to-back, the flawed mask update logic allows Port 0 to regain priority before Ports 1, 2, and 3 are serviced, causing indefinite starvation (**Liveness failure**).

I will run arbiter_baseline.sv through Synopsys VC Formal / Cadence JasperGold and record baseline numbers: passing vs. failing properties, Counterexample (CEX) cycle depths, reachable state counts, and assertion coverage percentages.

3.3 Planned Improvement

Using the exact same EDA tool settings, Tcl scripts, and bound SVA checker to ensure a fair comparison, I will implement the improved **Thermometer-Masked Dual-Path Arbiter (**arbiter_improved.sv**)**:

1. **Breaking the Combinational Loop:** I will decouple the next-state mask computation (mask_next) from the current-cycle grant logic by registering mask_q[3:0] strictly on the rising clock edge (posedge clk).
2. **Dual-Path Masked/Unmasked Priority Resolution:** I will implement two parallel priority encoders:
    - A **Masked Priority Resolver** operating on req_masked = req & mask_q.
    - An **Unmasked Priority Resolver** operating directly on req.
    - If any masked request is active (|req_masked), the arbiter issues the masked grant (gnt_masked); otherwise, a multiplexer immediately falls back to the unmasked grant (gnt_unmasked) in the exact same clock cycle. This guarantees work-conserving behavior (zero deadlocks) and strict round-robin rotation (zero starvation).

3.4 The Trade-off

Achieving 100% Liveness and Safety closure introduces clear hardware and verification trade-offs that I will quantify in the final report:

- **Hardware Area vs. Fairness (Starvation Freedom):** Compared to a basic single-path or fixed-priority arbiter, the improved dual-path Round-Robin architecture requires an extra 4-bit priority encoder, a 4-bit state register (mask_q), and a 2:1 output multiplexer. I am trading a modest increase in combinational gate count and flip-flop area for guaranteed fairness and deadlock-free operation.
- **Formal Proof Runtime (Safety vs. Liveness State-Space Exploration):** Proving Safety properties (Gp) only requires forward reachability analysis, which completes in a fraction of a second. Proving Liveness properties ($G(p \rightarrow Fq)$) requires the model checker to search for lasso-shaped fair loops across the Kripke structure, trading increased formal engine runtime and memory usage for complete mathematical certainty against livelocks.

4. Planned Approach and Formal Verification Test Plan

I will develop a non-intrusive SystemVerilog Assertions module (arbiter_sva.sv) bound to the DUV via the bind statement. The formal verification test plan consists of **12 properties** across four verification phases:

4.1 Environmental Assumptions (Input Constraints)

To prevent bogus liveness counterexamples where a master drops its request before the arbiter can grant it, I will define formal fairness assumptions for each port $i \in \{0,1,2,3\}$: asm_req_hold[i]: If a master asserts req[i] and is not granted (!gnt[i]), it must keep req[i] asserted in the next clock cycle (req[i] && !gnt[i] |=> req[i]).

4.2 Property Verification Matrix (Baseline vs. Expected Target)

Property ID	Verification Mode	Formal Property Description	Expected Baseline	Target After Improvement
1. aep_combo_loop	AEP Analysis	Structural check proving zero combinational feedback loops exist in the RTL.	**FAIL (Loop Found)**	**PASS (0 Violations)**
2. aep_x_prop	AEP / FXP	Prove no reachable state drives unknown (X) values onto gnt[3:0].	**PASS**	**PASS (Proven)**
3. ast_reset_state	Safety (FPV)	During active-low reset (!rst_n), gnt == 4'b0000 and mask_q == 4'b1111.	**PASS**	**PASS (Proven)**
4. ast_mutex_grant	Safety (FPV)	Mutual Exclusion: At most one port is granted per cycle ($onehot0(gnt)).	**PASS**	**PASS (Proven)**
5. ast_valid_grant	Safety (FPV)	No spurious grant: A port is granted only if requesting ((gnt & ~req) == 4'b0000).	**PASS**	**PASS (Proven)**
6. ast_no_deadlock	Safety (FPV)	Deadlock Freedom: If any request is active (|req), a grant must issue (|gnt).	**FAIL (CEX Trace)**	**PASS (Proven)**

Property ID	Verification Mode	Formal Property Description	Expected Baseline	Target After Improvement
7. ast_rr_rotation	Safety (FPV)	Priority Rotation: A granted port cannot win immediately on the next cycle if others wait.	**FAIL (CEX Trace)**	**PASS (Proven)**
8. ast_live_port0	Liveness (LTL)	Starvation Freedom Port 0: req[0] |-> s_eventually(gnt[0]).	**PASS**	**PASS (Proven)**
9. ast_live_port1	Liveness (LTL)	Starvation Freedom Port 1: req[1] |-> s_eventually(gnt[1]).	**FAIL (Lasso CEX)**	**PASS (Proven)**
10. ast_live_port2	Liveness (LTL)	Starvation Freedom Port 2: req[2] |-> s_eventually(gnt[2]).	**FAIL (Lasso CEX)**	**PASS (Proven)**
11. ast_live_port3	Liveness (LTL)	Starvation Freedom Port 3: req[3] |-> s_eventually(gnt[3]).	**FAIL (Lasso CEX)**	**PASS (Proven)**
12. cov_full_rr	Reachability Cover	Witness trace showing consecutive grants 0001 -> 0010 -> 0100 -> 1000 when req == 4'b1111.	**Unreachable (0%)**	**Covered (100%)**

5 Tools

- **Design & Verification Languages:** SystemVerilog (IEEE 1800) and SystemVerilog Assertions (SVA).
- **Formal Verification EDA Tools: Synopsys VC Formal** and **Cadence JasperGold** hosted on the Portland State University ECE Linux servers.
- **Scripting & Version Control:** Tcl automation scripts (run_baseline.tcl, run_improved.tcl) for single-command batch execution, and GitHub for repository hosting (shared with songx@pdx.edu and pratibha@pdx.edu).